

Class based and functional Based component

1 Class-Based Components

What are they?

Class-based components are ES6 classes that extend `React.Component` and use a `render()` method to return JSX.

Syntax

```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}

export default Welcome;
```

Key Features

- Use `this.props` and `this.state`
 - State is managed using `this.setState()`
 - Use lifecycle methods (`componentDidMount`, etc.)
 - More verbose
-

State Example

```
class Counter extends Component {
  state = { count: 0 };

  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <>
        <h1>{this.state.count}</h1>
        <button onClick={this.increment}>+</button>
      </>
    );
  }
}
```

Lifecycle Methods (Important)

- `componentDidMount()`
 - `componentDidUpdate()`
 - `componentWillUnmount()`
-

2 Functional Components

What are they?

Functional components are JavaScript functions that return JSX.

Originally stateless, but now with Hooks, they can manage state and lifecycle.

Syntax

```
function Welcome({ name }) {  
  return <h1>Hello, {name}</h1>;  
}  
  
export default Welcome;
```

State with Hooks

```
import { useState } from "react";  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <>  
      <h1>{count}</h1>  
      <button onClick={() => setCount(count +  
1)}>+</button>  
    </>  
  );  
}
```

Lifecycle with Hooks

```
useEffect(() => {  
  console.log("Mounted");  
  
  return () => {  
    console.log("Unmounted");  
  };  
}, []);
```

3 Class vs Functional Components (Interview ★)

Feature	Class Component	Functional Component
Syntax	ES6 class	Function
State	<code>this.state</code>	<code>useState</code>
Lifecycle	Lifecycle methods	<code>useEffect</code>
Code size	More	Less
Performance	Slightly slower	Better
<code>this</code> keyword	Required	Not needed
Recommended	✗ Old	✓ Modern

Which one should you use today?

👉 Functional Components ✓

Because:

- Simpler syntax

- Hooks replace lifecycle methods
 - Better performance
 - Official recommendation by React
-

Interview-ready Answer

Class-based components use ES6 classes and lifecycle methods, while functional components use functions and hooks to manage state and side effects. Functional components are now the preferred approach in React.

One-line Answer

Functional components are simpler and use hooks, while class components use lifecycle methods and `this.state`.

Hinglish Explanation

Class components purane tareeke se state aur lifecycle handle karte hain, jabki functional components hooks ka use karke simple aur efficient hote hain.

Common Interview Question

Q: Are class components deprecated?

  No, but not recommended for new projects

If you want next:

- 🔥 Lifecycle methods vs Hooks mapping
- 🔥 Why hooks cannot be used in classes
- 🔥 Functional component interview questions

Just tell me 😊

what do you mean by life cycle here and what these method Lifecycle Methods (Important) componentDidMount() componentDidUpdate() componentWillUnmount()

Great question 👍

This is a core React concept, so let's understand it slowly, clearly, and interview-ready.

What do we mean by “Lifecycle” in React?

Lifecycle refers to the different phases a component goes through from the time it is:

- 1 Created
- 2 Rendered on screen
- 3 Updated
- 4 Removed from the screen

👉 Just like a human life cycle: birth → growth → death

React Component Lifecycle Phases

1 Mounting (Birth)

Component is created and added to the DOM.

2 Updating (Growth)

Component re-renders due to:

- State change
- Props change

3 Unmounting (Death)

Component is removed from the DOM.

Lifecycle Methods (Class Components)

These are special methods that run automatically at specific stages.

1 `componentDidMount()`

When does it run?

✓ Once, after the component is first rendered to the DOM.

Use cases:

- API calls
- Subscriptions
- Timers
- DOM manipulation

Example

```
class User extends React.Component {  
  componentDidMount() {  
    console.log("Component mounted");  
  }  
}
```

```
render() {  
  return <h1>Hello User</h1>;  
}  
}
```

 Runs after render, not before.

2 `componentDidUpdate()`

When does it run?

✓ After every update (state or props change).

Use cases:

- Responding to prop changes
- API calls based on updates
- Syncing data

Example

```
class Counter extends React.Component {  
  componentDidUpdate(prevProps, prevState) {  
    if (prevState.count !== this.state.count) {  
      console.log("Count updated");  
    }  
  }  
}  
  
state = { count: 0 };  
  
render() {  
  return (  
    <>
```



```

        <h1>{this.state.count}</h1>
        <button onClick={() => this.setState({ count:
this.state.count + 1 })}>
            +
        </button>
    </>
    );
}
}

```

📌 Always compare previous values to avoid infinite loops.

3 `componentWillUnmount()`

When does it run?

✅ Just before the component is removed from the DOM.

Use cases:

- Cleanup
- Remove event listeners
- Clear timers
- Cancel subscriptions

Example

```

class Timer extends React.Component {
  componentWillUnmount() {
    console.log("Component unmounted");
    clearInterval(this.timer);
  }
}

```

```

render() {
  return <h1>Timer</h1>;
}
}

```

Lifecycle Methods Summary Table ★

Method	When it runs	Purpose
componentDidMount	After first render	API calls, setup
componentDidUpdate	After updates	Respond to changes
componentWillUnmount	Before removal	Cleanup

Functional Component Equivalent (Hooks) 🔄

Class Method	Hook
componentDidMount	<code>useEffect(() => {}, [])</code>
componentDidUpdate	<code>useEffect(() => {}, [deps])</code>
componentWillUnmount	<code>useEffect(() => { return () => {} }, [])</code>

Interview-ready Answer

The React component lifecycle describes the stages a component goes through from creation to removal, and lifecycle methods allow developers to run code at specific stages like mounting, updating, and unmounting.

One-line Answer

Lifecycle methods control what happens when a component mounts, updates, and unmounts.

Hinglish Explanation

Lifecycle ka matlab hota hai component ka janm, update aur destroy hone ke phases, aur lifecycle methods un phases me code chalane ke liye use hote hain.

Common Interview Question

Q: Are lifecycle methods available in functional components?

  No

  Replaced by Hooks (**useEffect**)

If you want next:

-  Full lifecycle diagram
-  useEffect deep dive
-  Common lifecycle mistakes

- 🔥 Interview Q&A on lifecycle

Just tell me 😊